EXP NO:07 DATE:	KOHONEN'S SELF-ORGANIZING MAP (SOM)

AIM:

To implement Kohonen's Self-Organizing Map (SOM) for clustering and visualizing highdimensional data using the Iris dataset.

ALGORITHM:

1. Install and import required libraries (NumPy, Pandas, Matplotlib, MiniSom, scikitlearn).
2. Load the Iris dataset.
3. Preprocess the data by normalizing/standardizing it so all features contribute equally.
4. Initialize the SOM with a grid of neurons, each having randomly initialized weights, and set the learning rate.
5. Train the SOM:
   - For each input vector, calculate the squared Euclidean distance to every neuron's weight vector: $D(j) = \Sigma(wij - xi)^2$ for i=1 to n, j=1 to m.
   - Find the neuron with minimum distance — this is the "winning" neuron (Best Matching Unit).
   - Update the winning neuron's weights (and its neighbors') to move closer to the input: $wij(new) = wij(old) + \alpha[xi - wij(old)]$.
   - Gradually reduce the learning rate over iterations: $\alpha(t+1) = 0.5 \times t$.
   - Repeat until the stopping condition (max iterations) is reached.
6. Visualize the results by plotting the SOM grid, marking which class label "wins" at each grid position.
7. Analyze the clusters formed to see how the SOM has organized similar data points close together on the 2D grid.

CODE:

```
import numpy as np import pandas as
pd import matplotlib.pyplot as plt
from minisom import MiniSom from
sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load dataset data = load_iris() df =
pd.DataFrame(data.data, columns=data.feature_names)

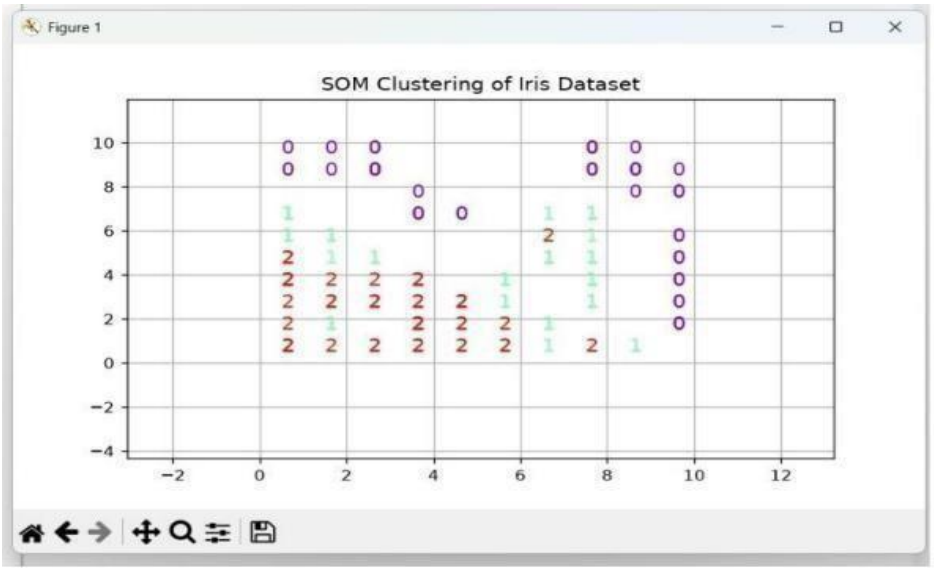
# Preprocess data scaler
= StandardScaler()
df_scaled = scaler.fit_transform(df)
```

```
# Initialize and train the SOM
som = MiniSom(x=10, y=10, input_len=df_scaled.shape[1], sigma=1.0,
learning_rate=0.5) som.random_weights_init(df_scaled) som.train_random(df_scaled,
500)  # Train with 500 iterations

# Visualize the results
plt.figure(figsize=(10, 10)) for
i, x in enumerate(df_scaled):
    w = som.winner(x)  # Get the winning node for each data point
plt.text(w[0] + 0.5, w[1] + 0.5, str(data.target[i]),
             color=plt.cm.rainbow(data.target[i] / 2), fontdict={'size': 12})

plt.xlim([0, 10]) plt.ylim([0, 10])
plt.grid() plt.title('SOM Clustering of
Iris Dataset') plt.show()
```

OUTPUT:



CLASS PERFORMANCE	
RECORD	
VIVA	
TOTAL	

RESULT:

Thus, Kohonen's Self-Organizing Map was successfully implemented on the Iris dataset, demonstrating a clustering approach that maps high-dimensional data onto a 2D grid while preserving the topological relationships between data points of different classes.

EXP NO:08 DATE:	PRINCIPAL COMPONENT ANALYSIS (PCA)

AIM:

To implement Principal Component Analysis (PCA) technique to reduce the dimensionality of the dataset.

ALGORITHM:

1. Import necessary libraries.
2. Load the dataset (Iris dataset).
3. Standardize the features so all variables contribute equally regardless of their original scale.
4. Apply PCA, transforming the standardized data into a smaller number of principal components that capture the maximum variance.
5. Check correlation between the resulting principal components to confirm they are uncorrelated (a key property of PCA).

CODE:

```
# Import necessary libraries from sklearn import
datasets import pandas as pd from
sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import seaborn as sns import
matplotlib.pyplot as plt
import numpy as np

# Load the Dataset
iris = datasets.load_iris()

# convert the dataset into a pandas data frame
df = pd.DataFrame(iris['data'], columns=iris['feature_names'])

# display the head (first 5 rows) of the dataset
print(df.head())

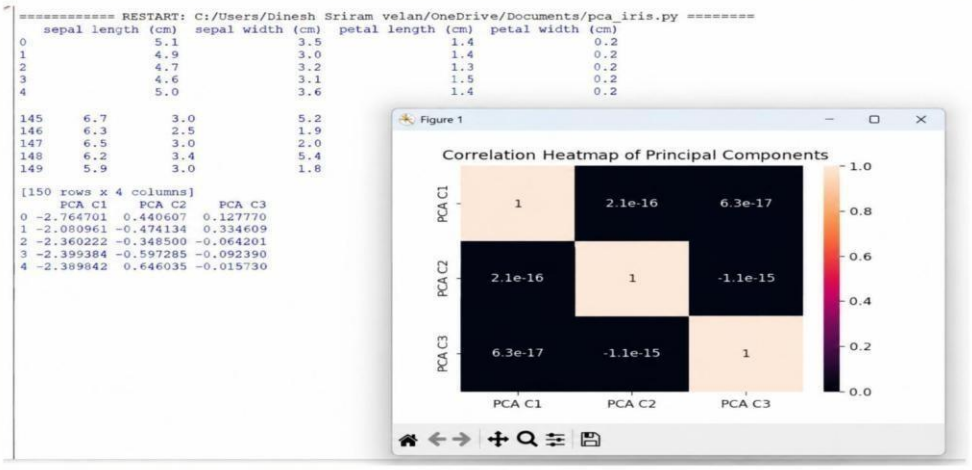
# Standardize the features scalar = StandardScaler()
scaled_data =
pd.DataFrame(scalar.fit_transform(df))
print(scaled_data)

# Applying PCA pca =
PCA(n_components=3)
pca.fit(scaled_data)
```

```
data_pca = pca.transform(scaled_data)
data_pca = pd.DataFrame(data_pca, columns=['PCA C1', 'PCA C2', 'PCA C3'])
print(data_pca.head())

# Checking Co-relation between features after PCA
data_pca_corr = data_pca.corr()
sns.heatmap(data_pca_corr, annot=True)
plt.title('Correlation Heatmap of Principal Components')
plt.show()
```

OUTPUT:



CLASS PERFORMANCE	
RECORD	
VIVA	
TOTAL	

RESULT:

Thus, Principal Component Analysis (PCA) was successfully implemented to reduce the dimensionality of the Iris dataset from 4 features down to 3 principal components. The heatmap confirms there is no correlation between the resulting principal components (PC1, PC2, PC3), demonstrating that PCA achieved a lower-dimensional representation while preserving independence between the transformed features.

AD.EX:1A Date:	IMPLEMENTATION OF NEURAL NETWORKS USING TENSORFLOW

AIM

To implement a simple neural network using TensorFlow and perform basic classification.

ALGORITHM

- Import TensorFlow and NumPy.
- Create binary classification data.
- Create a neural network model.
- Add a Dropout layer to reduce overfitting.
- Compile the model.
- Use Early Stopping during training.
- Train the model and display the accuracy.

PROGRAM

```
import tensorflow as tf
import numpy as np

x = np.array([1, 2, 3, 4, 5, 6]) y
= np.array([0, 0, 0, 1, 1, 1])

model = tf.keras.Sequential([     tf.keras.layers.Dense(1,
activation='sigmoid', input_shape=[1])
])

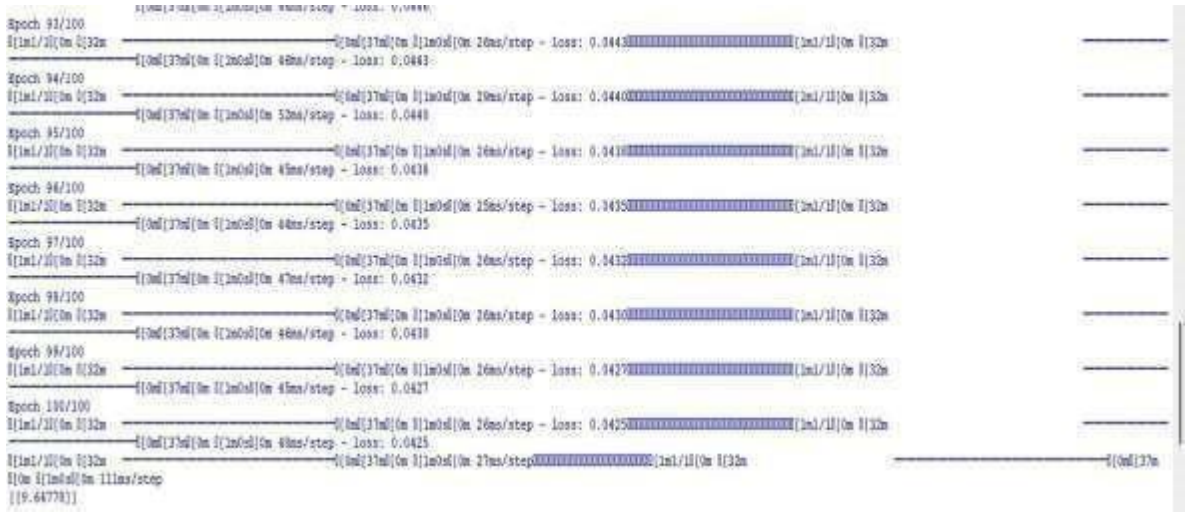
model.compile(
optimizer='adam',
loss='binary_crossentropy',
   metrics=['accuracy']
)

model.fit(x, y, epochs=100, verbose=0)

prediction = model.predict(np.array([5]))

print(prediction)
```

OUTPUT



Class Performance	
Record	
Viva	
Total	
Signature	

RESULT

Thus, a simple neural network was successfully created and trained using TensorFlow.

AD.EX:1B Date:	IMPLEMENTATION OF BINARY CLASSIFICATION USING TENSORFLOW

AIM

To perform binary classification using TensorFlow.

ALGORITHM

- Import TensorFlow and NumPy.
- Create input data with two classes: 0 and 1.
- Create a simple neural network.
- Compile the model.
- Train the model.
- Predict the class of new data.

PROGRAM

```
import tensorflow as tf
import numpy as np

x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype=float)
y = np.array([0, 0, 0, 0, 1, 1, 1, 1, 1, 1], dtype=float)

model = tf.keras.Sequential([
tf.keras.Input(shape=(1,)),
   tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
optimizer='adam',
loss='binary_crossentropy',
   metrics=['accuracy']
)

model.fit(x, y, epochs=500, verbose=0)

hours = float(input("Enter study hours: "))

prediction = model.predict(np.array([hours]), verbose=0)

if prediction[0][0] >= 0.5:
   print("Pass")
else:
print("Fail")
print("Pass probability:", prediction[0][0])
```

OUTPUT

```
===== RESTART: C:/Users/user/AppData/Local/Programs/Python/Python313/k.py ====
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the Tens
orFlow-DirectML plugin.
Enter study hours: 8
Pass
Pass probability: 0.6424905
```

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT

Thus, the Binary Classification model was successfully implemented using TensorFlow, and the model predicted the output as Pass (Class 1) for the given input**.**

AD.EX:1C Date:	BINARY CLASSIFICATION USING DROPOUT AND EARLY STOPPING IN TENSORFLOW

AIM

To perform binary classification using TensorFlow with Dropout and Early Stopping.

ALGORITHM

- Import TensorFlow and NumPy.
- Create input data x and binary output data y with classes 0 and 1.
- Create a neural network using Sequential.
- Add a Dense layer with ReLU activation.
- Add a Dropout layer to reduce overfitting.
- Add an output Dense layer with Sigmoid activation for binary classification.
- Compile the model using Adam optimizer and binary cross-entropy loss.
- Create Early Stopping to stop training when the loss stops improving.
- Train the model for a maximum of 100 epochs.
- Predict the class for the input value 7.
-

PROGRAM

```
import tensorflow as tf
import numpy as np

x = np.array([1, 2, 3, 4, 5, 6, 7, 8]) y
= np.array([0, 0, 0, 0, 1, 1, 1, 1])

model = tf.keras.Sequential([     tf.keras.layers.Dense(16,
activation='relu', input_shape=[1]),
tf.keras.layers.Dropout(0.2),      tf.keras.layers.Dense(1,
activation='sigmoid')
])
model.compile(
optimizer='adam',
loss='binary_crossentropy',
   metrics=['accuracy']
)
stop = tf.keras.callbacks.EarlyStopping(
   monitor='loss',
   patience=5
)
model.fit(
   x, y,
   epochs=100,     callbacks=[stop],     verbose=0)
prediction = model.predict(np.array([7]), verbose=0)
print("Prediction:", prediction)
```

OUTPUT

```
========== RESTART: C:/Users/user/AppData/Local/Programs/Python/Python313/k.py ==========

Warning (from warnings module):
  File "C:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\core\dense.py", line 107
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the Tens
orFlow-DirectML plugin.
Prediction: [[0.5823564]]
```

Class Performance	
Record	
Viva	
Total	
Signature	

RESULT

Thus, binary classification was successfully performed using TensorFlow with Dropout and Early Stopping.

AD.EX:2 Date:	DIMENSIONALITY REDUCTION USING ISOMAP

AIM

To perform dimensionality reduction on the S-Curve dataset using the Isomap algorithm and visualize the original and reduced data.

ALGORITHM

- Import make_s_curve from Scikit-learn to generate the S-Curve dataset.
- Generate 1000 samples using make_s_curve().
- Create an Isomap model with:
    1. n_neighbors = 10
    2. n_components = 2
- Apply Isomap using fit_transform() to reduce the data to 2 dimensions.
- Plot the original S-Curve data.
- Plot the 2D data obtained after Isomap dimensionality reduction.
- Display both plots for comparison.

PROGRAM

```
from sklearn.datasets import make_s_curve from
sklearn.manifold import Isomap
import matplotlib.pyplot as plt

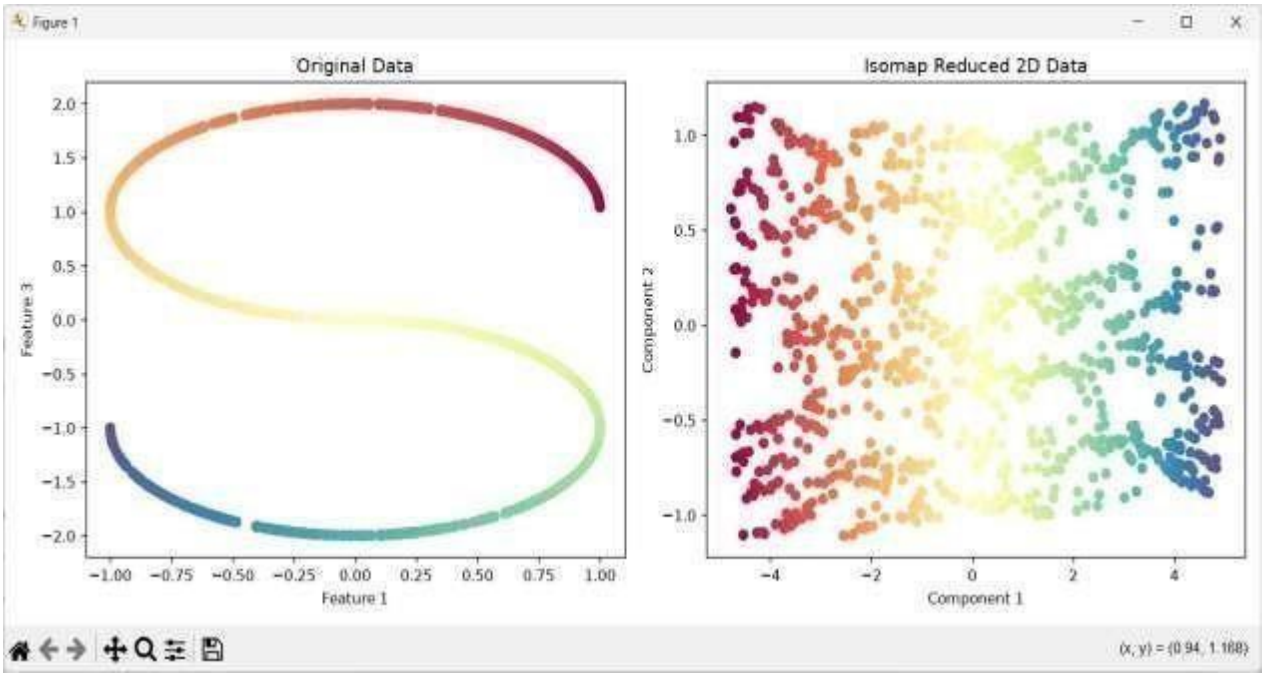
# Generate S-Curve dataset
X, color = make_s_curve(n_samples=1000, random_state=42)
# Create Isomap model
isomap = Isomap(n_neighbors=10, n_components=2)

# Reduce dimensions
X_isomap = isomap.fit_transform(X)

# Create plots
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# Original data
ax[0].scatter(X[:, 0], X[:, 2], c=color, cmap=plt.cm.Spectral)
ax[0].set_title("Original Data") ax[0].set_xlabel("Feature
1") ax[0].set_ylabel("Feature 3")
# Reduced data ax[1].scatter(X_isomap[:, 0],
X_isomap[:, 1],          c=color,
cmap=plt.cm.Spectral)
ax[1].set_title("Isomap Reduced 2D Data")
ax[1].set_xlabel("Component 1")
ax[1].set_ylabel("Component 2")
plt.tight_layout() plt.show()
```

OUTPUT



Class Performance	
Record	
Viva	
Total	
Signature	

RESULT

The S-Curve dataset was successfully reduced from 3 dimensions to 2 dimensions using the Isomap algorithm, and the original and reduced data were visualized successfully.

AD.EX:3 Date:	Dimensionality Reduction using Locally Linear Embedding (LLE)

AIM:

To perform dimensionality reduction on the Swiss Roll dataset using the Locally Linear Embedding (LLE) algorithm and visualize the original and reduced data.

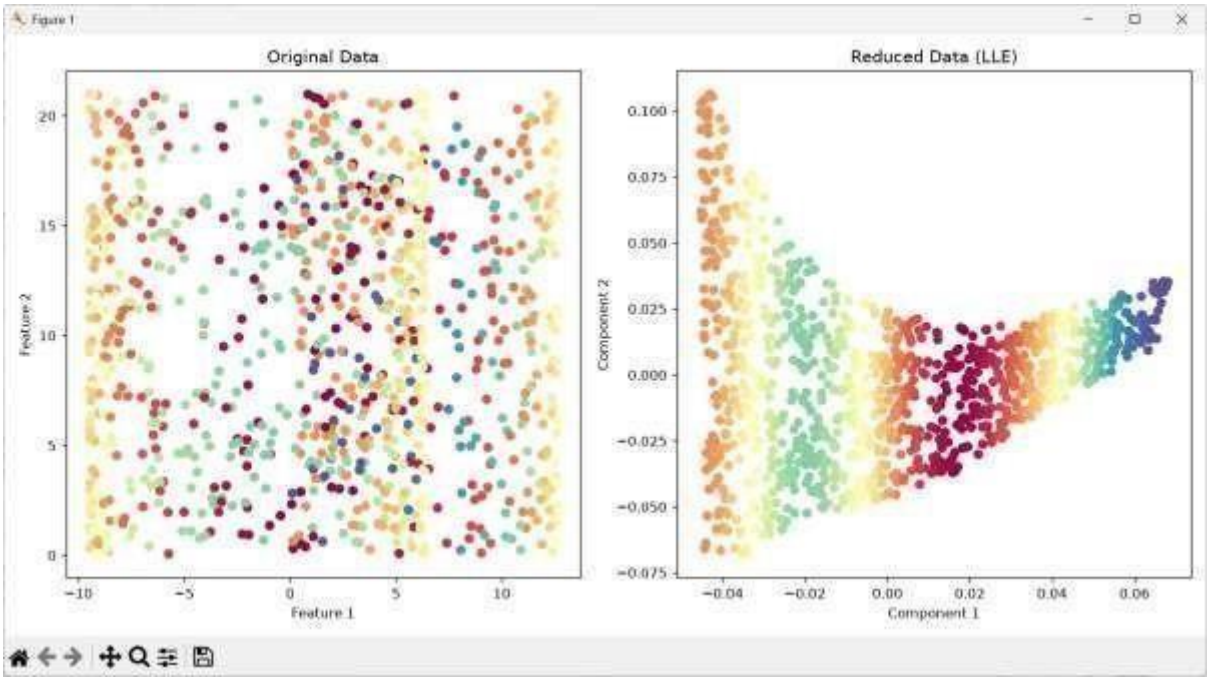
ALGORITHM:

- Import NumPy, Matplotlib, and the required Scikit-learn libraries.
- Generate a Swiss Roll dataset with 1000 samples.
- Set the number of nearest neighbors to 10.
- Create an LLE model with 2 components.
- Apply LLE to reduce the 3D data into 2D.
- Plot the original Swiss Roll data.
- Plot the reduced 2D data obtained using LLE.
- Display both plots for comparison.

PROGRAM

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_swiss_roll from
sklearn.manifold import LocallyLinearEmbedding
# Set parameters n_samples
= 1000 n_neighbors = 10
# Generate Swiss Roll dataset
X, color = make_swiss_roll(n_samples=n_samples)
# Create LLE model lle =
LocallyLinearEmbedding(
n_neighbors=n_neighbors,
n_components=2
)
X_reduced = lle.fit_transform(X) plt.figure(figsize=(12,
6))
# Original data plt.subplot(1, 2, 1) plt.scatter(X[:, 0], X[:,
1], c=X[:, 2], cmap=plt.cm.Spectral) plt.title("Original
Data") plt.xlabel("Feature 1") plt.ylabel("Feature 2") #
Reduced data plt.subplot(1, 2, 2) plt.scatter(X_reduced[:,
0], X_reduced[:, 1], c=X[:, 2], cmap=plt.cm.Spectral)
plt.title("Reduced Data (LLE)") plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.tight_layout() plt.show()
```

OUTPUT



Class Performance	
Record	
Viva	
Total	
Signature	

RESULT

The Swiss Roll dataset was successfully reduced from 3 dimensions to 2 dimensions using the LLE algorithm, and the original and reduced data were visualized successfully.

SRI SHAKTHI

INSTITUTE OF ENGINEERING AND TECHNOLOGY

COIMBATORE – 62

Autonomous Institution, Accredited by NAAC with "A" Grade

POWERING THE YOUTH
EMPOWERING THE NATION

21CS521 DATA SCIENCE AND ANALYTICS LABORATORY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRI SHAKTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY- COIMBATORE – 62

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

21CS521– DATA SCIENCE AND ANALYTICS

LABORATORY RECORD

NAME……..

ROLL NO : …….

CLASS : ……..…….

BRANCH:…….……...

ACADEMIC YEAR:……..

BATCH:……..……SEMESTER:……

Certified and Bonafide record of work done by…….……..

Place:

Date:

Staff In-Charge **Head of the Department**

University Register Number …….....

Submitted for the University Practical Examination held on……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

INDEX

Exp.No	DATE	NAME OF THE EXPRIEMENT	MARKS	SIGNATURE
1		Download, install and explore the features of NumPy, SciPy, Jupyter, Statsmodels and Pandas packages.		
2		Pre-process the dataset to remove missing values, outliers and to transform and Discretize the dataset		
3		Implement the classification techniques: Decision Tree and Naïve Bayes algorithms and compare its performance.		
4(a)		Implement K-Means clustering algorithm		
4(b)		Implement K-Medoids clustering algorithms		
5		Implement Apriori algorithm to derive association rules.		
6		Classify the input dataset using Support Vector Machine (SVM)		
7		Demonstrate clustering approach using Kohonen's self-organizing map		
8		Implement Principal Component Analysis (PCA) technique to reduce the dimensionality of the dataset		

ADDITIONAL EXPERIMENTS

Exp.No	DATE	NAME OF THE EXPRIEMENT	MARKS	SIGNATURE
A1.1		Implementation of Neuralnetwork using Tensorflow for Iris Flower Classification		
A1.2		Student Pass/Fail Prediction using Tensorflow Binary Classification		
A1.3		Binary Classification Using Dropout and Early Stopping in Tensorflow		
A2		Dimensionality Reductio using LocallyLinearEmbedding(LLE)		
A3		Dimensionality Reduction using Isomap		
A4		Sale Performance Analysis using PowerBI		
A5		Discount & Profit Analysis using Tableau		